

SOFTWARE REQUIREMENTS SPECIFICATION

AI-Powered Fraud Detection System

for Digital Banking Transactions

Hybrid Detection Architecture: Rule-Based Engine, Machine Learning Models, and AI Ensemble/Decisioning Layer

Document Type: Software Requirements Specification (SRS)

Conforms to the structure of IEEE 830 / ISO/IEC/IEEE 29148 SRS guidance

Version: 1.0 — Draft

Date: June 2026

All factual claims about industry practice, regulation, and architecture in this document are drawn from the sources listed in Section 9 (References). Figures used as illustrative targets are marked as such and should be validated against the organization's own data before being treated as commitments.

Table of Contents

Table of Contents.....	2
1. Introduction	5
1.1 Purpose	5
1.2 Scope	5
1.3 Definitions, Acronyms, and Abbreviations.....	5
1.4 References	6
1.5 Overview	6
2. Overall Description.....	7
2.1 Product Perspective	7
2.2 Product Functions (Summary)	7
2.3 User Classes and Characteristics	7
2.4 Operating Environment.....	8
2.5 Design and Implementation Constraints	8
2.6 Assumptions and Dependencies	8
3. Specific Requirements	9
3.1 Rule-Based Fraud Detection Engine.....	9
3.1.1 Description and Rationale	9
3.1.2 Functional Requirements	9
3.1.3 Rationale for Combinable, Configurable Rules.....	10
3.2 Machine Learning Detection Engine	10
3.2.1 Model Portfolio	10
3.2.2 Feature Engineering Requirements.....	11
3.2.3 Training, Evaluation, and Deployment Requirements.....	11
3.2.4 Illustrative Reference Points (Not Commitments)	12
3.3 AI Ensemble and Risk-Decisioning Engine	12
3.3.1 Functional Requirements	12
3.3.2 Explainability Requirement — Rationale	13
3.4 Shared Modules (Fed by All Three Detection Layers).....	13
3.4.1 Case Management.....	13
3.4.2 Alerting.....	13
3.4.3 Reporting and Compliance	14
4. External Interface Requirements	15
4.1 User Interfaces	15
4.2 System Interfaces.....	15
4.3 Communications Interfaces	15

5. Non-Functional Requirements.....	16
5.1 Performance	16
5.2 Security	16
5.3 Regulatory Compliance	16
5.4 Auditability and Explainability.....	16
5.5 Availability and Reliability	17
6. Data Requirements	18
7. System Architecture (Informative)	19
7.1 Reference Architecture Mapping	19
7.2 End-to-End Transaction Flow.....	19
7.3 Continuous Learning Loop	19
8. Assumptions, Dependencies, and Open Items.....	20
A. Fraud Type Taxonomy and Detection Mapping	21
B. Use Cases and User Stories.....	24
B.1 Use Case: Real-Time Transaction Fraud Check	24
B.2 Use Case: Fraud Analyst Case Investigation	24
B.3 Use Case: Risk Officer Rule Configuration.....	25
B.4 Use Case: ML Model Retraining and Deployment	25
B.5 Use Case: Compliance Officer SAR Data Export	26
C. User Stories.....	27
C.1 Customer.....	27
C.2 Fraud Analyst	27
C.3 Risk Officer.....	27
C.4 Data Scientist / ML Engineer	27
C.5 Compliance Officer	28
D. Detailed Data Models	29
D.1 Transaction Event	29
D.2 Fraud Decision Record	30
D.3 Case Entity	30
D.4 Customer Risk Profile	31
D.5 ML Model Registry Entry.....	32
E. Feature Engineering Specification	33
E.1 Transaction-Level Features.....	33
E.2 Velocity Features (Rolling Window).....	33
E.3 Behavioral / Profile Features	34
E.4 Feature Store Requirement.....	35

F. Detailed Machine Learning Model Specifications	36
F.1 Supervised Gradient Boosting Models (XGBoost / LightGBM).....	36
F.2 Deep Neural Network (DNN)	36
F.3 LSTM (Long Short-Term Memory).....	37
F.4 Autoencoder (Anomaly Detection).....	38
F.5 Isolation Forest (Anomaly Detection).....	38
F.6 Graph-Based AML Detection.....	38
G. User Interface and Experience Requirements	40
G.1 Fraud Analyst Case Console	40
G.2 Risk Officer Rule Configuration Console	40
G.3 Executive Dashboard.....	41
H. Deployment and Infrastructure Requirements.....	42
H.1 Deployment Architecture.....	42
H.2 Event Streaming	42
H.3 Data Storage.....	42
H.4 Monitoring and Observability.....	43
H.5 Security and Network.....	43
I. Testing Requirements	45
I.1 Unit Testing	45
I.2 Integration Testing.....	45
I.3 ML Model Validation Testing.....	45
I.4 Performance and Load Testing.....	46
I.5 Security Testing.....	46
I.6 Fraud Type Coverage Testing	46
J. Performance Benchmarks and Acceptance Criteria	48
K. Glossary	50
9. References	52

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for an AI-Powered Fraud Detection System (referred to throughout as “the System”) intended for use by a retail/digital bank to monitor transactions across mobile banking, internet banking, ATM, POS, QR payment, card, and agency banking channels.

This document follows the section structure recommended by IEEE Std 830 for software requirements specifications, the long-standing reference guide for SRS content and quality, later superseded by ISO/IEC/IEEE 29148 [1].

The defining characteristic of this System, distinct from a single-method fraud filter, is that it is built around three cooperating detection layers, each documented in detail in Section 3:

1. A rule-based engine that applies deterministic, analyst-defined conditions to every transaction.
2. A machine learning layer that scores transactions using supervised and unsupervised models trained on historical data.
3. An AI ensemble/decisioning layer that combines the outputs of (1) and (2), along with contextual risk signals, into a single actionable decision.

1.2 Scope

The System monitors digital banking transactions in real time or near-real time, assigns each transaction a fraud risk classification, and triggers an automated response (approve, hold, reject, or freeze) consistent with that classification. It also provides supporting capabilities for fraud analyst case review, alerting, model lifecycle management, and regulatory/compliance reporting.

The System is designed around patterns documented in real deployed architectures — including AWS's published reference architecture for bank fraud detection, which combines a managed ML fraud-detection service with a rules-based analytics layer and a graph database for anti-money-laundering pattern detection [2] — rather than a single proprietary design. Section 7 (System Architecture) maps the System's components against this reference pattern.

Out of scope for this document: core banking ledger functionality, payment switching/settlement logic, and the bank's general identity and access management platform, except where the System must integrate with them.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
------	------------

AML	Anti-Money Laundering — regulatory framework requiring banks to detect and report transactions linked to laundering of illicit funds [3][4].
ATO	Account Takeover — fraud in which an attacker gains unauthorized control of a customer account using stolen credentials, phishing, or social engineering [2].
FPR	False Positive Rate — proportion of flagged transactions that are subsequently confirmed legitimate.
KYC	Know Your Customer — identity verification and risk profiling performed at onboarding and on an ongoing basis [5].
PCI DSS	Payment Card Industry Data Security Standard — security standard for systems that store, process, or transmit cardholder data [6].
SAR	Suspicious Activity Report — formal report filed with financial intelligence authorities (e.g., FinCEN in the US) when suspected fraud or money laundering is identified [6].
SHAP	SHapley Additive exPlanations — a model-explainability technique used to attribute a model's output to individual input features [6][7].
SRS	Software Requirements Specification.
XGBoost / LightGBM	Gradient-boosted decision tree algorithms widely used as supervised fraud-classification models [8][9].

1.4 References

Full citation details for all numbered references ([1]–[14]) appear in Section 9. In-text citations throughout this document correspond to that reference list.

1.5 Overview

Section 2 describes the System at a high level: context, user classes, constraints, and assumptions. Section 3 is the core of this specification — it defines requirements separately for each of the three detection approaches (rule-based, machine learning, and AI ensemble), plus the shared requirements for case management, alerting, and reporting. Section 4 covers external interface requirements. Section 5 specifies non-functional requirements (performance, security, compliance). Section 6 covers data requirements. Section 7 documents system architecture. Section 8 lists assumptions and dependencies. Section 9 is the reference list.

2. Overall Description

2.1 Product Perspective

The System is a component within the bank's broader digital banking ecosystem. It does not replace the core banking system, payment switch, or KYC/onboarding platform; it sits alongside them, consuming transaction events and customer/account context, and returning a risk decision before or shortly after a transaction is committed.

This product perspective mirrors deployed industry patterns: published architectures position the fraud detection system as an event-driven layer that receives transaction and session events through an API gateway, evaluates them against ML models and business rules, and publishes a decision to a fraud event bus for downstream consumption by case management and notification systems [2].

2.2 Product Functions (Summary)

- Real-time and batch transaction monitoring across digital banking channels.
- Three-layer fraud detection: rule-based engine, machine learning models, and an AI ensemble/decision layer (full detail in Section 3).
- Risk-based decisioning: approve, approve-with-monitoring, hold for review, reject, or freeze account.
- Case management workflow for fraud analysts.
- Alerting via SMS, email, and push notification.
- Dashboards and scheduled/regulatory reporting.
- Model lifecycle management: training, evaluation, deployment, monitoring, and retraining.

2.3 User Classes and Characteristics

User Class	Description / Interaction with the System
Customer	Initiates transactions; receives alerts and step-up verification requests when a transaction is held.
Fraud Analyst	Reviews flagged transactions and cases; confirms or dismisses fraud; the System's case management module (Section 3.4) is built for this role.
Risk Officer	Sets and tunes rule thresholds and risk-score bands used by the rule engine and decision logic.
Compliance Officer	Consumes AML/KYC-relevant outputs and regulatory reports; responsible for SAR filing decisions [4][6].
Data Scientist / ML Engineer	Trains, evaluates, deploys, and retrains the machine learning models in Section 3.2.

System Administrator	Manages access control, configuration, and platform availability.
-----------------------------	---

2.4 Operating Environment

The System is designed to be cloud-deployable (AWS, Azure, or GCP) or run on-premises where data residency requirements apply, consistent with the hybrid on-premise/cloud deployment pattern many financial institutions use to balance sensitive-data handling with the scalability of managed AI services [10].

2.5 Design and Implementation Constraints

- Must comply with PCI DSS for any component that touches cardholder data [6].
- Must support AML/KYC obligations, including ongoing transaction monitoring and the ability to generate data needed for Suspicious Activity Reports [3][4][6].
- Risk thresholds, rule definitions, and model score cutoffs must be configurable by Risk Officers without requiring an engineering release, a capability identified as a core architectural requirement in regulated fintech systems [5].
- Model outputs used to decline or freeze a customer transaction must be explainable at the feature level (see SHAP requirement, Section 3.3) to support audit and customer dispute resolution [6][7].

2.6 Assumptions and Dependencies

- The bank has, or will provide, a minimum volume of historically labeled fraud/non-fraud transactions sufficient to train an initial supervised model; exact volume requirements are addressed in Section 8.
- Core banking and channel systems can emit transaction events in near-real time (directly or via a streaming/ETL bridge).
- Existing identity, KYC, and watchlist data sources are accessible to the System under the bank's data governance policy.

3. Specific Requirements

This section specifies requirements for each of the System's three cooperating fraud-detection approaches, followed by the shared modules (decisioning, case management, alerting, reporting, model management) that all three feed into. Each requirement is identified for traceability, consistent with IEEE 830 guidance that requirements be uniquely identifiable and traceable through design and test [1].

Industry practice does not treat these three approaches as competitors; published guidance and deployed systems consistently combine them, since rule engines catch known patterns instantly while machine learning catches patterns no one has written a rule for yet, and the combination outperforms either alone [11][12][13].

3.1 Rule-Based Fraud Detection Engine

The rule-based engine applies deterministic, human-authored conditions to every transaction and login event. It is the System's fastest detection layer and the one most directly auditable by Risk and Compliance, since each decision traces to an explicit, named condition [11].

3.1.1 Description and Rationale

Rule-based engines remain a core layer in modern fraud platforms because they evaluate transactions in real time against clearly defined conditions, can be created or modified by risk teams without an engineering release, and give an immediately auditable reason for every flag — properties that are harder to get from a pure ML model [11]. Their main limitation, well documented in the literature, is that fraudsters can study and route around fixed thresholds, and rules alone do not adapt to fraud patterns that did not exist when the rule was written [12][13]. The System therefore uses the rule engine as the first, fast-decisioning layer and relies on Section 3.2/3.3 to catch what rules miss.

3.1.2 Functional Requirements

Req ID	Requirement
RB-1	The System shall evaluate every transaction against the active rule set before, or concurrently with, ML scoring, and shall complete this evaluation within the latency budget defined in Section 5.1.
RB-2	The System shall support, at minimum, the following rule categories drawn from common deployed fraud rule sets [2][11]: velocity rules (e.g., multiple transactions within seconds, multiple failed logins), threshold rules (transaction amount exceeds configured daily/per-transaction limit), geographic rules (high-risk country, impossible travel — e.g., two logins from different countries within an implausible time window), device/session rules (device mismatch, new device combined with new ISP), and account-state rules (blacklisted account, dormant account suddenly active).

RB-3	The System shall allow a Risk Officer to create, modify, disable, or retire a rule, and to adjust its threshold values, through a configuration interface that does not require a code deployment.
RB-4	The System shall support combinable rule conditions (AND/OR logic) so that compound real-world patterns can be expressed, for example: “customer logged in from two or more different countries within two hours AND the ISP is new to this customer” — a pattern documented as a real account-takeover rule in deployed bank fraud architectures [2].
RB-5	The System shall log, for every rule evaluation, which rule(s) fired, their configured threshold at the time of evaluation, and the resulting action, to support audit and dispute resolution.
RB-6	The System shall provide a sandbox/shadow mode in which a new or modified rule can be evaluated against live or replayed traffic without affecting production decisions, before being promoted to production — a safeguard explicitly recommended for rule-engine change management [11].
RB-7	The System shall version every rule set change with timestamp and author, and shall support rollback to a prior rule set version.

3.1.3 Rationale for Combinable, Configurable Rules

Rule engines used in production fraud platforms are explicitly designed for risk teams to adjust thresholds and add rules in response to emerging fraud trends without engineering intervention; this agility is treated as a primary advantage of the rule-engine layer, not an optional nicety [11][5].

3.2 Machine Learning Detection Engine

The ML engine produces a continuous fraud-probability score for each transaction by learning from historical transaction patterns, customer behavior, and (where available) confirmed fraud labels. This is the layer that addresses the core weakness of rules alone: the ability to recognize fraud patterns that were never explicitly programmed [12][13].

3.2.1 Model Portfolio

Published fraud-detection research and deployed systems consistently favor a portfolio of models rather than a single algorithm, because different model families catch different fraud signatures and combining them improves both accuracy and robustness to adversarial adaptation [9][14][29]. The System's model portfolio:

Category	Algorithms	Role / Rationale
Supervised — Gradient Boosting	XGBoost, LightGBM	Trained on labeled historical fraud/non-fraud transactions; strong baseline accuracy and reasonably interpretable feature importances; used as the base learner in multiple published hybrid fraud systems [8][9][14].

Supervised — Classical	Logistic Regression, Random Forest	Used as benchmark/comparison models and, in ensemble designs, as meta-learners that combine other models' outputs [9][14].
Deep Learning	Deep Neural Network, LSTM, Autoencoder	DNNs capture non-linear feature interactions that boosting models can miss; autoencoders and LSTMs are used to flag anomalies in sequences of transactions [29][14].
Unsupervised / Semi-supervised	Isolation Forest, One-Class SVM, DBSCAN, Autoencoder	Used where labeled fraud examples are scarce; establishes a behavioral baseline and flags deviations, which is the same approach behind managed services such as Amazon Fraud Detector's Account Takeover Insights model, designed specifically to work without requiring collected fraud labels [2].

3.2.2 Feature Engineering Requirements

Req ID	Requirement
ML-1	The System shall compute, for each transaction, features including: transaction amount, transaction frequency, average daily spend, time-of-day, geolocation, device ID, IP address/ISP, merchant category, login time, recent failed login attempts, account age, transaction velocity, and distance from the customer's previous transaction location — a feature set consistent with those used in production banking fraud-check APIs [2].
ML-2	The System shall maintain a feature store that serves identical feature computation logic to both the model training pipeline and the real-time scoring pipeline, to prevent training/serving skew.
ML-3	The System shall support behavioral-baseline features derived from a customer's own transaction history (not just population-wide statistics), consistent with semi-supervised approaches that learn normal behavior per user [2].

3.2.3 Training, Evaluation, and Deployment Requirements

Req ID	Requirement
ML-4	The System shall support training on historical labeled transaction data, with documented handling of class imbalance (fraud is always a small minority class), using techniques such as class weighting, focal loss, or threshold optimization rather than naive resampling alone, consistent with current best practice for production fraud models [29].
ML-5	The System shall evaluate every candidate model on precision, recall, F1-score, and AUC-ROC on a held-out test set before deployment; accuracy alone shall not be treated as a sufficient evaluation metric, since accuracy is misleading under severe class imbalance [30][31].
ML-6	The System shall maintain a model registry recording each trained model's version, training data window, evaluation metrics, and deployment status.

ML-7	The System shall support shadow deployment of a new model (scoring live traffic without influencing production decisions) before promotion to production, mirroring the staged-rollout approach financial institutions use when introducing new ML models alongside existing systems [16].
ML-8	The System shall monitor deployed model performance (score distribution drift, precision/recall on newly confirmed fraud cases) on an ongoing basis and shall alert the ML engineering team when performance degrades beyond a configured tolerance.
ML-9	The System shall support scheduled and on-demand retraining using newly confirmed fraud/non-fraud labels sourced from the case management module (Section 3.4).

3.2.4 Illustrative Reference Points (Not Commitments)

The figures below are reported results from third-party published studies on fraud datasets, included to show what is achievable with these techniques. They are not guaranteed outcomes for this System and must not be used as contractual targets until validated against the organization's own data.

- A cost-sensitive ensemble framework evaluated on real-world online banking transaction data reported catching approximately 98% of actual fraud cases (recall), outperforming static rule-only baselines on highly imbalanced data [19].
- A stacking ensemble combining decision trees, random forest, SVM, XGBoost, CatBoost, and logistic regression reported an F1-score of 88.14% on a credit card fraud dataset, balancing precision and recall better than any single base model [30][31].
- A hyper-ensemble combining six unsupervised/semi-supervised model types, evaluated on a real-world dataset of bank transactions from a large European bank, reported base-learner accuracies in the 0.91–0.97 range, with the ensemble outperforming individual baselines [33].

3.3 AI Ensemble and Risk-Decisioning Engine

This layer is what distinguishes a fraud platform from a fraud filter: it does not pick a winner between the rule engine and the ML models, it combines their outputs — along with customer risk profile, AML risk score, device risk, and geolocation risk — into a single risk classification and decision. This hybrid approach is the pattern most consistently recommended in both vendor architectures and academic literature, since rule outputs and ML outputs catch different things and combining them outperforms relying on either alone [2][12][13][29].

3.3.1 Functional Requirements

Req ID	Requirement
AI-1	The System shall compute a composite risk score for each transaction by combining: (a) the rule engine's triggered-rule outcomes, (b) the ML engine's fraud-probability score(s), (c) the customer's existing risk profile/category, (d) an AML risk indicator, and (e) device and geolocation risk signals — the same composite-input pattern used in deployed risk-scoring engines [2].
AI-2	The System shall classify each transaction into one of four risk bands: Low, Medium, High, or Critical, based on the composite score and configurable band boundaries.

AI-3	The System shall map each risk band to a default automated decision: Low → Approve; Medium → Approve with monitoring; High → Hold for review; Critical → Reject or Freeze account — with the mapping itself configurable by Risk Officers.
AI-4	Where the rule engine and ML engine disagree (e.g., no rule fires but the ML score is high, or a low-severity rule fires but the ML score is low), the System shall apply a documented resolution policy rather than silently favoring one signal; the default policy shall be configurable (e.g., “take the higher of the two implied risk levels”).
AI-5	The System shall produce a SHAP-based or equivalent feature-attribution explanation for every transaction routed to Hold, Reject, or Freeze, so that a fraud analyst, auditor, or customer-facing agent can see which features and rules drove the decision — a requirement directly tied to explainability obligations in regulated financial environments [6][7][29].
AI-6	The System shall log the full decision trail (rule outcomes, model score(s), composite score, band, and final decision) for every transaction, retained per the audit retention period in Section 5.4.
AI-7	The System shall support step-up authentication or hold-for-verification as an intermediate action for Medium/High risk transactions, rather than only binary approve/reject, consistent with proportionate-response designs used in production risk engines [29].

3.3.2 Explainability Requirement — Rationale

Explainability is not a convenience feature for this layer; published implementations explicitly tie SHAP-based interpretability to audit-trail and due-diligence obligations under PCI DSS v4.0 and AML directives, since each transaction's risk assessment needs to be demonstrable, not just accurate [6]. A model that cannot explain a Critical-risk decision cannot be defended in a customer dispute or regulatory examination.

3.4 Shared Modules (Fed by All Three Detection Layers)

3.4.1 Case Management

Req ID	Requirement
CM-1	The System shall generate a case automatically for every transaction routed to Hold, Reject, or Freeze, pre-populated with the decision trail from AI-6.
CM-2	The System shall allow a Fraud Analyst to assign, annotate, escalate, approve, reject, or close a case, and to request additional customer verification.
CM-3	The System shall feed analyst-confirmed fraud/non-fraud outcomes back into the ML training pipeline (ML-9) as new labeled data.

3.4.2 Alerting

Req ID	Requirement
--------	-------------

AL-1	The System shall notify the customer via SMS, email, or push notification (per their preference) when a transaction is held, rejected, or requires step-up verification.
AL-2	The System shall notify assigned Fraud Analysts of new high/critical-risk cases through an internal alert and/or case-management ticket.

3.4.3 Reporting and Compliance

Req ID	Requirement
RP-1	The System shall generate daily, weekly, and monthly fraud summary reports covering transaction volume, fraud rate, false-positive rate, and losses avoided.
RP-2	The System shall produce data in the format needed to support Suspicious Activity Report (SAR) filing obligations under applicable AML regulation (e.g., the U.S. Bank Secrecy Act and FinCEN requirements, or jurisdictional equivalents) [3][4][6].
RP-3	The System shall provide an executive dashboard showing real-time KPIs: total transactions, fraud transactions, fraud rate, high-risk accounts, false-positive rate, active alerts, and model accuracy trend.

4. External Interface Requirements

4.1 User Interfaces

- Fraud Analyst case review console (Section 3.4.1).
- Risk Officer rule and threshold configuration console (Section 3.1.2, RB-3).
- Executive/management dashboard (Section 3.4.3, RP-3).

4.2 System Interfaces

The System exposes a synchronous fraud-check API, invoked by channel applications (mobile, web, ATM/POS gateway) during transaction or login processing, following the integration pattern used in deployed bank fraud-check APIs: the calling application submits event attributes (customer/entity ID, IP address, device fingerprint, session ID, transaction details) and receives a risk decision in response [2].

Representative input attributes, consistent with those used in production fraud-check APIs [2]:

- entity_id (customer or account identifier)
- ip_address, device fingerprint, session_id
- transaction amount, currency, merchant category, beneficiary identifier
- geolocation / country code

4.3 Communications Interfaces

- Real-time event streaming (e.g., Kafka or an equivalent managed event bus) for transaction ingestion and decision publishing.
- SMS and email gateway integration for customer alerting.
- Integration with the bank's core banking system, KYC/AML platform, and credit bureau or watchlist data sources.

5. Non-Functional Requirements

5.1 Performance

ID	Requirement
NF-1	The System shall return a risk decision for real-time channel transactions (mobile, POS, card) with sub-second latency under normal load, consistent with latency expectations for synchronous fraud-check APIs used in production [2].
NF-2	The System shall scale to the bank's peak transaction-per-second volume without degraded latency; managed fraud-detection services in comparable deployed architectures are documented as auto-scaling to handle on the order of 200+ predictions per second, with higher throughput available on request [2].

5.2 Security

ID	Requirement
NF-3	Any System component that stores, processes, or transmits cardholder data shall comply with PCI DSS [6].
NF-4	The System shall encrypt sensitive customer and transaction data at rest and in transit.
NF-5	The System shall enforce role-based access control (RBAC) and multi-factor authentication for all internal user roles defined in Section 2.3.

5.3 Regulatory Compliance

ID	Requirement
NF-6	The System shall support the ongoing transaction monitoring obligations required under AML regulation, including the U.S. Bank Secrecy Act and equivalent frameworks in other jurisdictions [3][4].
NF-7	The System shall retain decision logs, model predictions, and rule evaluation records for the audit retention period required by applicable regulation and the bank's own record-retention policy.
NF-8	The System shall support a risk-based approach to customer monitoring consistent with KYC/CDD (Customer Due Diligence) expectations, including the ability to apply enhanced monitoring to higher-risk customer segments [5].

5.4 Auditability and Explainability

ID	Requirement
----	-------------

NF-9	Every automated Hold, Reject, or Freeze decision shall be reconstructable after the fact from logged rule outcomes, model scores, and the SHAP-based feature attribution required by AI-5.
-------------	--

5.5 Availability and Reliability

ID	Requirement
NF-10	The System shall define and meet an availability target appropriate to a production payment-adjacent system (commonly 99.9% or higher in comparable financial infrastructure); the exact target should be set jointly with IT Operations and is not fixed by this document.
NF-11	The System shall fail safe: if the fraud engine is unreachable or times out, the calling channel application's fallback behavior (e.g., default to existing rule-only checks, or apply a conservative hold policy) shall be explicitly defined rather than left to default to silent approval.

6. Data Requirements

The quality and structure of training data has more influence on real-world model performance than algorithm choice; this section specifies what the System needs from the bank's data, not just what the models compute.

- Historical transaction data, with confirmed fraud/non-fraud labels where available, sufficient to train and validate the initial supervised models in Section 3.2.
- Customer profile data: account type, risk category, account age, registered beneficiaries, device history.
- Session/device data: device fingerprint, IP address, login timestamps, failed login attempts.
- Data must be handled under the bank's data governance and privacy policy; anonymization or pseudonymization should be applied wherever the data leaves a production system boundary for model training, consistent with data-protection obligations such as GDPR where applicable [5][21].
- Severe class imbalance should be expected and explicitly planned for (fraud is typically a small fraction of total transactions); see ML-4 for the System's required handling approach.

7. System Architecture (Informative)

This section is informative rather than strictly normative: it shows one architecture capable of satisfying Section 3's requirements, modeled directly on a published reference architecture for bank fraud detection [2], rather than prescribing a specific vendor.

7.1 Reference Architecture Mapping

The AWS published reference architecture for banking fraud detection organizes the system into five building blocks: event injection, event storage, fraud detection (ML + rules + decision engine), event orchestration, and fraud response [2]. The table below maps this System's requirements onto that same block structure.

Reference Block [2]	Realized By
Event Injection	Channel applications call the fraud-check API (Section 4.2) with transaction/session attributes.
Event Store	Persists raw events for both real-time scoring and downstream model retraining.
Fraud Detection	Rule engine (3.1) + ML models (3.2) + AI ensemble/decision engine (3.3), running in parallel and feeding the composite score.
Event Orchestration	Publishes the decision and full decision trail to an event bus for downstream consumption.
Fraud Response	Case management (3.4.1), alerting (3.4.2), and channel-level enforcement of the decision (approve/hold/reject/freeze).

7.2 End-to-End Transaction Flow

```

Customer -> Banking Channel -> API Gateway -> Authentication
-> Event Bus -> [ Rule Engine | ML Models ] -> AI Ensemble / Risk Scoring
-> Decision Engine -> [ Approve | Hold | Reject | Freeze ]
-> Alerting -> Case Management -> Fraud Analyst Dashboard

```

7.3 Continuous Learning Loop

Continuous retraining is treated in deployed systems as integral to the architecture, not an optional add-on, since fraud patterns evolve and a static model degrades [2][17]:

```

Historical + Newly Confirmed Cases -> Feature Engineering -> Model Training
-> Evaluation -> Model Registry -> Shadow Deployment -> Production
-> Performance Monitoring -> Retraining Trigger

```

8. Assumptions, Dependencies, and Open Items

The following items should be resolved or validated before or during implementation; they are flagged here rather than answered, since the answers depend on the specific bank's data and infrastructure.

4. Volume and quality of historical labeled fraud data available for initial model training (affects which models in Section 3.2.1 are viable at launch — unsupervised/semi-supervised methods may need to lead if labels are scarce, per the rationale in ML-4 and the Amazon Fraud Detector ATI precedent [2]).
5. Target jurisdiction(s) and corresponding AML/KYC regulatory regime(s) (the requirements in Section 5.3 reference US/FinCEN and general AML/KYC practice as illustrative; jurisdiction-specific legal review is required before this is treated as a compliance commitment).
6. Specific latency and availability SLAs (Section 5.1, 5.5), to be set jointly with IT Operations and channel owners.
7. Cloud vs. on-premises vs. hybrid deployment decision, which affects which managed AI/ML services (if any) are used to realize Section 3.2 and 3.3.
8. Fallback behavior when the fraud engine is unavailable (NF-11) must be explicitly decided by Risk and Operations, not defaulted.

A. Fraud Type Taxonomy and Detection Mapping

Before specifying detection requirements in detail, this section catalogues the principal fraud types the System must be capable of detecting, grounded in fraud typologies documented across banking industry publications and regulatory guidance. For each type, the table identifies which of the three detection layers (Rule-Based = RB, Machine Learning = ML, AI Ensemble = AI) is primarily responsible for detection, and what signals are used.

This taxonomy is essential for test planning: each fraud type in Section A must correspond to at least one test scenario in Section 10 (Testing Requirements).

Fraud Type	Description	Real-World Reference	Primary Detection Layer(s)	Key Signals
Account Takeover (ATO)	Attacker uses stolen credentials, phishing, or social engineering to gain control of a customer account and initiates unauthorized transactions.	AWS deployed architecture explicitly names ATO as a primary fraud type addressed by its ML-based fraud detection service [2].	RB + ML + AI	New device, new ISP, new country, failed logins before success, impossible travel, new beneficiary added.
Card-Not-Present (CNP) Fraud	Fraudulent use of card credentials (number, CVV, expiry) in online/remote transactions where the physical card is not required.	Extensively studied in published ML fraud literature; the widely-used Kaggle/ULB credit card fraud dataset reflects CNP fraud patterns [9][30].	ML + AI	Merchant category, transaction amount deviation from customer average, card-not-present flag, geolocation mismatch.
SIM Swap Fraud	Attacker convinces mobile carrier to transfer victim's phone number to attacker-controlled SIM, enabling OTP interception.	Identified as a specific detectable fraud signal in banking fraud rule-engine specifications [original content, section 3.1].	RB + ML	OTP delivery to new SIM indicator, login from different device within short window, mobile number change followed by high-value transaction.

Impossible Travel	Two authenticated sessions or transactions originate from geographically distant locations within a timeframe that precludes physical travel.	Named as a standard rule in deployed bank fraud rule sets and in the AWS reference architecture sample rules [2][11].	RB	Haversine distance between consecutive login/transaction geolocations divided by elapsed time exceeds configurable speed threshold.
Velocity Fraud	Multiple rapid transactions (often micro-transactions to test card validity, then a large transaction) within a short time window.	Documented as a standard velocity rule type in deployed fraud rule engines [11].	RB + ML	Transaction count per minute/hour, transaction amount sequence pattern, merchant diversity in short window.
Dormant Account Reactivation	A long-inactive account suddenly initiates high-value or unusual transactions, often indicating account takeover or synthetic identity fraud.	Named as a specific rule trigger in banking fraud rule-engine documentation [11].	RB + ML	Days since last transaction, transaction amount relative to pre-dormancy average, device/IP mismatch.
Money Mule / AML Layering	Legitimate customer account used (willingly or unwillingly) to receive and rapidly re-transfer funds, obscuring illicit origins.	Identified as a core AML fraud type requiring graph-based detection (Amazon Neptune used for AML pattern detection in AWS reference architecture) [2][3][4].	ML + AI (graph patterns)	Rapid in-and-out transfer pattern, beneficiary network analysis, transaction structuring (amounts just below reporting thresholds).
Synthetic Identity Fraud	Attacker creates a fabricated identity combining real and fictitious information, establishes credit/account, then commits fraud.	Documented in AML/KYC compliance literature as a growing threat requiring	ML + AI	Identity element inconsistency, application pattern matching known synthetic profiles,

		enhanced CDD [5].		behavioral baseline deviation from account age.
Business Email Compromise (BEC) / Authorized Push Payment (APP)	Victim is socially engineered into authorizing a large payment to a fraudster-controlled account.	Identified in AML transaction monitoring literature as a high-value fraud type increasingly targeted by AI monitoring [4].	ML + AI	First-time beneficiary, beneficiary account age, transaction amount significantly above customer average, timing patterns.
Insider Fraud	Bank employee or contractor misuses system access to alter records, approve unauthorized transactions, or exfiltrate data.	Requires behavioral analytics on internal user actions rather than only customer transaction patterns.	ML + AI (behavioral analytics on staff)	Unusual access hours, bulk data queries, approvals outside normal workflow, transactions on accounts linked to the employee.

The fraud types above represent the minimum coverage the System must address. The test plan (Section 10) shall include at least one test scenario per row in this table.

B. Use Cases and User Stories

This section documents the principal use cases the System must support, structured in the style recommended for SRS use-case documentation [1]. Each use case is linked to the requirements in Section 3 that implement it.

B.1 Use Case: Real-Time Transaction Fraud Check

Use Case ID	UC-01
Title	Real-Time Transaction Fraud Check
Primary Actor(s)	Customer (initiating a transaction); System (automated)
Preconditions	Customer is authenticated on a digital banking channel. Transaction details are available. The System is online and all three detection layers are active.
Main Success Flow	1. Customer initiates a transaction (e.g., mobile transfer). 2. Channel application invokes the fraud-check API with transaction and session attributes (Section 4.2). 3. System runs the rule engine (Section 3.1) against the transaction; zero or more rules fire. 4. System computes ML feature vector (Section 3.2, ML-1/ML-2) and obtains fraud probability score(s). 5. AI ensemble layer (Section 3.3, AI-1) combines rule outcomes, ML score, and contextual risk signals into a composite risk score. 6. Decision engine (AI-2/AI-3) classifies the score into a risk band and maps it to a decision. 7. Decision is returned to the channel application; if Approve, transaction proceeds. If Hold/Reject/Freeze, the appropriate response is triggered. 8. Full decision trail is logged (AI-6). Alert sent if applicable (AL-1/AL-2).
Post-Conditions	Transaction is approved, held, rejected, or account frozen per the risk decision. Decision trail is persisted. Customer is notified if action is taken.
Alternative / Exception Flow	If the fraud engine is unavailable or times out, the channel application applies the fallback policy (NF-11). If multiple rules fire at conflicting severity levels, AI-4 resolution policy applies.

B.2 Use Case: Fraud Analyst Case Investigation

Use Case ID	UC-02
Title	Fraud Analyst Case Investigation
Primary Actor(s)	Fraud Analyst
Preconditions	A transaction has been routed to Hold or flagged as High/Critical risk. A case has been auto-created (CM-1). The analyst is authenticated with the Fraud Analyst role.

Main Success Flow	1. Analyst opens the case management console and views the pending case queue. 2. Analyst selects a case and reviews the auto-populated decision trail: rule(s) fired, ML score, SHAP feature attribution (AI-5), and transaction details. 3. Analyst reviews customer transaction history and device/location context. 4. Analyst decides: (a) Confirm fraud — reject transaction, optionally freeze account, escalate; (b) Dismiss — approve transaction, clear flag; (c) Request step-up — send customer a verification challenge. 5. Analyst records the outcome and rationale in the case. 6. Confirmed outcome is fed back to the ML retraining pipeline (CM-3, ML-9).
Post-Conditions	Case is closed with a documented outcome. If fraud confirmed, relevant systems (core banking, alerting) are notified. Outcome label is available for model retraining.
Alternative / Exception Flow	If customer cannot be reached for step-up verification within the configured SLA, the case escalates to the next-tier analyst or Risk Officer.

B.3 Use Case: Risk Officer Rule Configuration

Use Case ID	UC-03
Title	Risk Officer Rule Configuration
Primary Actor(s)	Risk Officer
Preconditions	Risk Officer is authenticated. A new fraud pattern has been identified (e.g., from case analysis or industry alert) that is not covered by existing rules.
Main Success Flow	1. Risk Officer accesses the rule configuration console (RB-3). 2. Risk Officer creates a new rule specifying: condition(s) (with AND/OR logic per RB-4), threshold values, risk severity level, and target action. 3. Risk Officer activates sandbox/shadow mode (RB-6) and runs the rule against replayed historical traffic to review false-positive impact before promotion. 4. Risk Officer adjusts thresholds as needed based on shadow-mode results. 5. Risk Officer promotes the rule to production. System versions the change (RB-7). 6. System begins evaluating all new transactions against the new rule.
Post-Conditions	New rule is live in production. Rule set is versioned. Shadow-mode evaluation results are archived.
Alternative / Exception Flow	If the risk officer determines the false-positive rate is unacceptable in shadow mode, the rule is held in draft or modified before promotion. Rollback to prior version is available (RB-7).

B.4 Use Case: ML Model Retraining and Deployment

Use Case ID	UC-04
Title	ML Model Retraining and Deployment

Primary Actor(s)	Data Scientist / ML Engineer
Preconditions	Performance monitoring (ML-8) has triggered a degradation alert, or a scheduled retraining cycle has begun. New labeled cases from the case management module (CM-3) are available.
Main Success Flow	1. ML Engineer retrieves newly labeled fraud/non-fraud cases from the training data store. 2. Feature engineering pipeline recomputes features for the new training window (ML-2). 3. Model training is run with updated data; class imbalance is handled per ML-4. 4. New candidate model is evaluated on held-out test set: precision, recall, F1, AUC-ROC (ML-5). 5. If metrics meet the configured threshold, the model is registered in the model registry (ML-6). 6. Model is deployed in shadow mode (ML-7); real-time scores are computed but not used for decisions. 7. Shadow performance is reviewed. On approval, the new model is promoted to production. 8. Prior model is retained in the registry for rollback if needed.
Post-Conditions	Production model is updated. New model version is registered with evaluation metrics. Prior model retained for rollback.
Alternative / Exception Flow	If the new model underperforms the current production model on key metrics, it is not promoted; the ML team is alerted and the investigation is logged.

B.5 Use Case: Compliance Officer SAR Data Export

Use Case ID	UC-05
Title	Suspicious Activity Report (SAR) Data Export
Primary Actor(s)	Compliance Officer
Preconditions	One or more confirmed or suspected fraud/AML cases meet the threshold for SAR filing under applicable regulation [3][4][6].
Main Success Flow	1. Compliance Officer accesses the reporting module (RP-2). 2. System provides a filtered view of confirmed fraud cases and high-risk AML-flagged transactions within a specified date range. 3. Compliance Officer selects cases for SAR export. 4. System generates a structured export containing the required transaction details, decision trail, and risk narrative in a format compatible with the SAR filing process. 5. Compliance Officer reviews and files the SAR with the relevant authority (e.g., FinCEN in the US, or the jurisdictional equivalent). 6. Filing is recorded in the audit log (NF-7).
Post-Conditions	SAR data is exported and filing is documented. Audit log entry is created.
Alternative / Exception Flow	If required data fields are missing for any case (e.g., customer identity information not available), the System flags the gap and the Compliance Officer determines how to proceed prior to filing.

C. User Stories

In addition to the formal use cases above, the following user stories capture requirements from the perspective of each user role in the format: As a [role], I want [capability] so that [benefit]. These are intended to complement the structured requirements in Section 3 and to guide sprint/iteration planning.

C.1 Customer

- As a customer, I want to receive an immediate SMS or push notification if my transaction is held or rejected, so that I know what happened and can take action quickly. (Implements AL-1)
- As a customer, I want legitimate transactions to be processed without unnecessary friction, so that fraud detection does not disrupt my day-to-day banking. (Drives the false-positive rate requirement in NF and the Section 3.3 composite scoring rationale)
- As a customer, I want a simple way to challenge a blocked transaction (e.g., a step-up OTP or callback), so that genuine transactions can be unblocked quickly. (Implements AI-7)

C.2 Fraud Analyst

- As a fraud analyst, I want to see why a transaction was flagged — which rule fired and which features drove the ML score — so that I can make a fast, informed investigation decision. (Implements AI-5, SHAP requirement)
- As a fraud analyst, I want a structured case queue prioritized by risk level, so that I focus on the highest-impact cases first. (Implements CM-1/CM-2)
- As a fraud analyst, I want the ability to see a customer's transaction history, device history, and recent alerts alongside the current case, so that I have full context without switching systems.
- As a fraud analyst, I want my investigation outcomes to feed back into the model, so that confirmed fraud cases improve future detection. (Implements CM-3, ML-9)

C.3 Risk Officer

- As a risk officer, I want to add or modify fraud detection rules without raising a change request to the engineering team, so that I can respond to new fraud patterns within hours, not weeks. (Implements RB-3)
- As a risk officer, I want to test a new rule against historical traffic before it goes live, so that I can estimate false-positive impact before affecting customers. (Implements RB-6)
- As a risk officer, I want a dashboard showing false-positive rate, fraud detection rate, and losses by channel, so that I can tune thresholds with data rather than guesswork. (Implements RP-3)

C.4 Data Scientist / ML Engineer

- As an ML engineer, I want a feature store that serves the same feature logic in training and scoring, so that training/serving skew does not degrade production model performance. (Implements ML-2)

- As an ML engineer, I want model evaluation to report precision, recall, F1, and AUC-ROC — not just accuracy — so that I can properly assess model quality on imbalanced data. (Implements ML-5)
- As an ML engineer, I want to deploy a new model in shadow mode before promoting it, so that I can validate live-traffic performance without risking customer impact. (Implements ML-7)
- As an ML engineer, I want automated monitoring alerts when model score distribution drifts significantly, so that model degradation is caught proactively. (Implements ML-8)

C.5 Compliance Officer

- As a compliance officer, I want every automated fraud decision to be logged with a full decision trail, so that I can reconstruct any decision for audit or regulatory review. (Implements AI-6, NF-7, NF-9)
- As a compliance officer, I want the System to provide structured data exports for SAR filings, so that I can meet regulatory reporting obligations efficiently. (Implements RP-2)
- As a compliance officer, I want the System to support risk-based customer monitoring tiers, so that enhanced due diligence is applied to higher-risk segments as required by KYC regulation. (Implements NF-8)

D. Detailed Data Models

This section specifies the principal data entities the System must store, their key attributes, and their relationships. These data models inform both the database schema design and the API contract.

D.1 Transaction Event

The Transaction Event is the core input entity. Every fraud evaluation begins with a transaction event received by the fraud-check API. The attribute set below is consistent with the input schema used in production fraud-check API designs [2].

Attribute	Type	Description	Required
transaction_id	UUID	Globally unique identifier for this transaction event.	Yes
entity_id	String	Customer or account identifier.	Yes
channel	Enum	Originating channel: MOBILE, INTERNET, ATM, POS, QR, CARD, AGENCY.	Yes
amount	Decimal	Transaction amount in the account's base currency.	Yes
currency	ISO 4217	Currency code (e.g., USD, ETB, EUR).	Yes
merchant_category_code	MCC String	ISO 18245 merchant category code for the payee.	No
beneficiary_id	String	Identifier of the recipient account or payee.	No
ip_address	IPv4/IPv6	IP address of the originating session.	No
device_fingerprint	String	Hashed device identifier.	No
session_id	UUID	Session identifier for the originating authentication session.	No
geolocation_lat	Decimal	Latitude of the transaction origin, if available.	No
geolocation_lon	Decimal	Longitude of the transaction origin, if available.	No
country_code	ISO 3166-1	Country of transaction origin.	No
isp	String	Internet service provider of the originating IP.	No
timestamp	ISO 8601	UTC timestamp of the transaction.	Yes

credentials_valid	Boolean	Whether the authentication step passed (useful for ATO detection) [2].	Yes
-------------------	---------	--	-----

D.2 Fraud Decision Record

For every transaction event processed, the System persists a Fraud Decision Record. This record is the primary artifact for audit (NF-9), analyst investigation (CM-1), and regulatory review (NF-7).

Attribute	Type	Description
decision_id	UUID	Unique identifier for this decision record.
transaction_id	UUID	Foreign key to the Transaction Event.
rule_outcomes	JSON Array	List of rules evaluated, with: rule_id, fired (bool), threshold_at_time, severity.
ml_scores	JSON Array	List of ML model outputs: model_id, model_version, fraud_probability (0–1).
composite_score	Decimal (0–1)	Weighted combination of rule outcomes and ML scores by the AI ensemble layer (AI-1).
risk_band	Enum	LOW, MEDIUM, HIGH, CRITICAL (AI-2).
decision	Enum	APPROVE, APPROVE_MONITOR, HOLD, REJECT, FREEZE (AI-3).
shap_values	JSON Object	Feature-level attribution values for the highest-weighted ML model (AI-5).
resolution_policy_applied	String	Which resolution policy was applied if rule and ML signals conflicted (AI-4).
evaluated_at	ISO 8601	UTC timestamp of the decision.
decision_latency_ms	Integer	End-to-end latency in milliseconds (for SLA monitoring, NF-1).

D.3 Case Entity

A Case is created automatically for every HOLD, REJECT, or FREEZE decision (CM-1) and is the primary object managed through the analyst workflow (CM-2).

Attribute	Type	Description
case_id	UUID	Unique case identifier.

decision_id	UUID	FK to the Fraud Decision Record that triggered this case.
transaction_id	UUID	FK to the originating Transaction Event.
assigned_analyst_id	String	Analyst currently assigned to the case (nullable until assigned).
status	Enum	OPEN, IN_REVIEW, PENDING_VERIFICATION, ESCALATED, CLOSED_FRAUD, CLOSED_LEGITIMATE.
analyst_notes	Text	Free-text notes added by the investigating analyst.
outcome_label	Enum	FRAUD, NOT_FRAUD, INCONCLUSIVE — recorded at case closure and used for ML retraining (CM-3).
created_at	ISO 8601	Timestamp of automatic case creation.
closed_at	ISO 8601	Timestamp of case closure.
escalation_tier	Integer	Escalation level (1 = analyst, 2 = senior analyst, 3 = Risk Officer).

D.4 Customer Risk Profile

The Customer Risk Profile aggregates the information used by the AI ensemble layer (AI-1) and the rule engine for context-sensitive scoring.

Attribute	Type	Description
entity_id	String	Customer/account identifier (PK).
risk_category	Enum	LOW, MEDIUM, HIGH, CRITICAL — set by KYC/onboarding and updated by the System.
aml_risk_score	Decimal	AML risk score from the bank's AML/KYC platform.
account_age_days	Integer	Days since account opening.
avg_daily_spend_30d	Decimal	Rolling 30-day average daily spend.
known_devices	JSON Array	Set of device fingerprints seen in the last 90 days.
known_countries	JSON Array	Set of country codes seen in the last 90 days.
registered_beneficiaries	JSON Array	Beneficiary IDs registered and used by the customer.
failed_login_count_24h	Integer	Failed login attempts in the last 24 hours.
last_transaction_at	ISO 8601	Timestamp of the most recent transaction (used for dormancy detection).

sim_swap_flag	Boolean	Whether the customer's registered mobile number has changed recently.
---------------	---------	---

D.5 ML Model Registry Entry

Attribute	Type	Description
model_id	UUID	Unique model identifier.
model_name	String	Human-readable name (e.g., XGBoost_v3_mobile).
algorithm	String	Algorithm type (XGBoost, LightGBM, LSTM, Isolation Forest, etc.).
training_data_window	Date Range	Start and end date of the training data used.
training_data_size	Integer	Number of records in the training set.
precision	Decimal	Precision on the held-out test set (ML-5).
recall	Decimal	Recall on the held-out test set (ML-5).
f1_score	Decimal	F1-score on the held-out test set (ML-5).
auc_roc	Decimal	AUC-ROC on the held-out test set (ML-5).
deployment_status	Enum	DRAFT, SHADOW, PRODUCTION, RETIRED.
promoted_at	ISO 8601	Timestamp of promotion to production (null if not yet promoted).
trained_by	String	ML engineer who trained this version.

E. Feature Engineering Specification

Feature engineering — deriving the right input signals from raw transaction and behavioral data — has more impact on model performance than algorithm selection in most production fraud systems [14]. This section specifies the full feature set the System must compute, organized by feature category.

E.1 Transaction-Level Features

Feature Name	Computation	Fraud Signal
amount_abs	Raw transaction amount.	High-value transactions carry higher risk; combined with customer baseline.
amount_zscore	$(\text{amount} - \text{customer_avg_amount_90d}) / \text{customer_std_amount_90d}$	How far this transaction deviates from the customer's own spending pattern.
amount_to_limit_ratio	$\text{amount} / \text{daily_transaction_limit}$	Transactions near or at daily limit may indicate limit-testing behavior.
merchant_risk_score	Lookup from merchant risk table by MCC code.	High-risk MCCs (e.g., cryptocurrency exchanges, money transfer, gambling) elevate risk.
is_first_time_beneficiary	1 if beneficiary_id not in customer's registered_beneficiaries, else 0.	First-time beneficiary transactions are a strong ATO and APP fraud signal [2].
is_new_device	1 if device_fingerprint not in customer's known_devices, else 0.	New device combined with high-value or first-time-beneficiary is a high-risk combination.
is_new_country	1 if country_code not in customer's known_countries, else 0.	Geolocation outside customer's known set.
transaction_hour	Hour of day (0-23) from transaction timestamp (UTC).	Fraud is disproportionately common at unusual hours for the customer.
is_weekend	1 if timestamp falls on a Saturday or Sunday, else 0.	Temporal anomaly feature.

E.2 Velocity Features (Rolling Window)

Velocity features count or sum transaction attributes over rolling time windows. They are the numerical expression of the velocity rules in Section 3.1 (RB-2) and are also key ML model inputs.

Feature Name	Window	Computation
tx_count_1min	1 minute	Count of transactions from this entity in the last 1 minute.
tx_count_5min	5 minutes	Count of transactions from this entity in the last 5 minutes.
tx_count_1h	1 hour	Count of transactions from this entity in the last 1 hour.
tx_count_24h	24 hours	Count of transactions from this entity in the last 24 hours.
amount_sum_1h	1 hour	Sum of transaction amounts from this entity in the last 1 hour.
amount_sum_24h	24 hours	Sum of transaction amounts from this entity in the last 24 hours.
unique_merchants_1h	1 hour	Count of distinct merchant/beneficiary IDs in the last 1 hour (card testing signal).
unique_devices_24h	24 hours	Count of distinct device fingerprints used in the last 24 hours.
unique_countries_24h	24 hours	Count of distinct countries in the last 24 hours.
failed_login_count_1h	1 hour	Failed login attempts in the last 1 hour (ATO precursor signal).

E.3 Behavioral / Profile Features

Feature Name	Computation
days_since_last_transaction	Days between this transaction and the customer's previous transaction (dormancy signal).
account_age_days	Days since account opening (young accounts carry higher risk in some fraud typologies).
avg_tx_amount_30d	Rolling 30-day mean transaction amount for this customer.
avg_tx_count_per_day_30d	Rolling 30-day mean daily transaction count.
haversine_distance_km	Great-circle distance in km between this transaction's geolocation and the customer's previous transaction's geolocation. Used for impossible-travel detection (RB-2).

travel_speed_kmh	haversine_distance_km / (hours_since_last_tx + epsilon). Trips exceeding a configurable speed threshold (e.g., 900 km/h) flag impossible travel.
beneficiary_first_seen_days	Days since the current beneficiary was first used by this customer (0 = new).
customer_risk_category_encoded	Numeric encoding of the customer's KYC risk category (LOW=0, MEDIUM=1, HIGH=2, CRITICAL=3).
sim_swap_days_ago	Days since the customer's SIM/mobile number was changed (0 if changed today; null if not recently changed).

E.4 Feature Store Requirement

All features above must be computed by a centralized Feature Store that serves identical computation logic to both the model training pipeline and the real-time scoring pipeline (ML-2). This prevents training/serving skew — a common cause of production model underperformance where a model trained on one feature computation produces systematically different scores in production if the serving pipeline computes features differently.

Req ID	Requirement
FE-1	The Feature Store shall provide sub-100ms feature retrieval latency for all rolling-window features during real-time transaction scoring (contributing to the overall latency budget in NF-1).
FE-2	The Feature Store shall maintain pre-computed customer-level aggregates (velocity counts, rolling averages) updated incrementally on each new transaction event, rather than recomputing from scratch on each scoring request.
FE-3	Any change to a feature's computation logic shall increment the feature version in the Feature Store and shall require retraining of all deployed models that consume that feature before the new computation is promoted to production.

F. Detailed Machine Learning Model Specifications

This section provides the detailed technical specification for each model category in Section 3.2.1, including input/output contracts, training data requirements, evaluation criteria, and production integration requirements.

F.1 Supervised Gradient Boosting Models (XGBoost / LightGBM)

Gradient boosting algorithms are the current industry standard baseline for tabular transaction fraud data, combining high accuracy with reasonable interpretability via feature importance and SHAP [7][14]. Multiple published production fraud systems use XGBoost or LightGBM as a primary or ensemble component [8][9][14][29].

Specification Item	Detail
Input	Feature vector from Section E (transaction-level + velocity + behavioral features). All numeric; categorical features one-hot encoded or target-encoded.
Output	Scalar fraud probability in [0, 1].
Target variable	Binary: 1 = confirmed fraud, 0 = confirmed legitimate (from case management outcomes, CM-3).
Class imbalance handling	Class weighting (scale_pos_weight in XGBoost) or focal loss; SMOTE alone is insufficient for production fraud data and should be avoided as the primary strategy (ML-4) [29][30].
Hyperparameter tuning	Grid search or Bayesian optimization over: learning rate, max depth, n_estimators, subsample, colsample_bytree. Cross-validation on time-based splits (not random splits, to prevent future-data leakage).
Evaluation metrics	Precision, Recall, F1-Score, AUC-ROC on held-out test set (ML-5). Operating threshold selected to meet the organization's preferred precision/recall trade-off, not defaulted to 0.5 [29].
Explainability	SHAP TreeExplainer — provides per-transaction feature attribution values with near-zero additional latency overhead for tree-based models [7].
Retraining frequency	At minimum, triggered when AUC-ROC on rolling confirmed cases drops below a configured threshold relative to the production baseline (ML-8/ML-9).

F.2 Deep Neural Network (DNN)

DNNs capture complex non-linear interactions between features that gradient boosting may miss, particularly in high-dimensional feature spaces. In hybrid ensemble designs, the DNN contributes a complementary score alongside the gradient boosting model [29].

Specification Item	Detail
Architecture	Fully connected feedforward network; recommended starting configuration: input layer → [256, 128, 64] hidden units with ReLU activation → sigmoid output. Layer count and width are hyperparameters to be tuned.
Input	Same feature vector as F.1; features normalized to zero mean and unit variance before DNN input.
Output	Scalar fraud probability in [0, 1].
Framework	TensorFlow or PyTorch (both have documented use in banking fraud detection [11]).
Training	Mini-batch gradient descent with Adam optimizer. Dropout regularization to prevent overfitting on small fraud class. Early stopping based on validation recall.
Serving latency	DNN inference shall be benchmarked in staging to ensure it does not violate the latency budget in NF-1 when combined with the other model inferences in the AI ensemble.
Explainability	SHAP DeepExplainer or Kernel SHAP — computationally heavier than TreeExplainer; should be pre-computed or sampled if latency is a constraint [7].

F.3 LSTM (Long Short-Term Memory)

LSTMs model sequences of transactions, capturing temporal patterns such as a card-testing sequence (multiple small transactions followed by a large one) that point-in-time models miss [18].

Specification Item	Detail
Input	Sequence of the last N transactions for the customer (N configurable; recommend N=10 as starting point), each represented as the feature vector from Section E.
Output	Scalar fraud probability for the current (Nth) transaction, based on the full sequence context.
Architecture	One or two LSTM layers → dense layer → sigmoid output. Bidirectional LSTM optional; increases compute but may improve recall on sequential patterns.
Padding	Sequences shorter than N (e.g., new customers) padded with a sentinel value; the model must be trained on padded sequences to handle this at inference time.
Use case fit	Particularly valuable for velocity/sequence fraud typologies: card testing, structuring (money laundering), and rapid-transfer mule patterns.

F.4 Autoencoder (Anomaly Detection)

Autoencoders are trained to reconstruct normal transaction patterns. Transactions that deviate significantly from learned normal behavior — measured by reconstruction error — are flagged as anomalous. This approach is valuable when labeled fraud data is scarce, since it requires only normal data to train [11][18].

Specification Item	Detail
Training data	Confirmed legitimate transactions only (no fraud examples required).
Input/Output	Feature vector from Section E → compressed latent representation → reconstructed feature vector. Output dimensionality equals input dimensionality.
Anomaly score	Mean Squared Error (MSE) between input and reconstruction. High MSE = high anomaly. Threshold for flagging is set on held-out normal data (e.g., 95th or 99th percentile of MSE on legitimate transactions).
Use case fit	Detecting novel fraud patterns not seen in training data — new typologies that no rule and no supervised model has been trained on. Acts as an early warning layer in the AI ensemble.
Output to ensemble	The reconstruction error (normalized to [0,1]) is passed as an anomaly signal into the AI ensemble layer (Section 3.3, AI-1).

F.5 Isolation Forest (Anomaly Detection)

Isolation Forest explicitly isolates anomalies rather than profiling normality, making it computationally efficient and well-suited to high-dimensional transaction data [11][33].

Specification Item	Detail
Input	Feature vector from Section E; particularly effective on transaction-level and velocity features.
Output	Anomaly score in [-1, 1] (scikit-learn convention). Scores closer to -1 indicate greater anomaly.
Training	Trained on a representative sample of all transactions (not only legitimate ones, unlike the Autoencoder); the forest learns which data points are easiest to isolate.
Ensemble integration	Anomaly score normalized and passed to the AI ensemble as an additional unsupervised signal alongside the supervised model scores.

F.6 Graph-Based AML Detection

Standard tabular ML models treat each transaction in isolation. Money laundering patterns often span networks of accounts (layering: funds pass through chains of mule accounts before extraction). Graph databases can traverse these networks and identify structuring patterns that tabular models cannot see. The AWS reference architecture uses Amazon Neptune (a graph database) specifically for this AML fraud detection use case [2].

Req ID	Requirement
GR-1	The System shall maintain a transaction graph in which nodes represent accounts/entities and edges represent transactions, updated in near-real time.
GR-2	The System shall run graph traversal queries to detect common AML typologies: layering chains (funds passing through N intermediate accounts within T hours), structuring (multiple transactions just below reporting thresholds), and fan-in/fan-out patterns (many senders to one receiver or one sender to many receivers in a short window).
GR-3	Graph-based AML risk signals shall be included as inputs to the AI ensemble layer (AI-1) for accounts flagged by graph analysis.
GR-4	Graph query latency shall not block real-time transaction scoring for channels requiring sub-second response; graph analysis results may be applied asynchronously and used to update the customer's risk profile (Section D.4) for subsequent transactions.

G. User Interface and Experience Requirements

This section specifies the requirements for the System's user-facing interfaces. Three primary interfaces are required: the Fraud Analyst Case Console, the Risk Officer Rule Configuration Console, and the Executive Dashboard.

G.1 Fraud Analyst Case Console

Req ID	Requirement
UI-1	The case queue shall display, at minimum, the following columns without requiring horizontal scrolling on a standard 1920×1080 display: Case ID, Transaction ID, Customer ID, Channel, Amount, Risk Band (color-coded: green/yellow/orange/red for LOW/MEDIUM/HIGH/CRITICAL), Decision, Created At, Status, Assigned Analyst.
UI-2	The queue shall support sorting by: Risk Band (default: highest first), Amount, Created At, and Status; and filtering by: Channel, Risk Band, Status, Date Range, and Assigned Analyst.
UI-3	Selecting a case shall open a detail view showing in a single screen without scrolling: (a) transaction attributes, (b) the triggered rules and their thresholds, (c) the ML model score(s), (d) the composite risk score and band, (e) the top 5 SHAP feature contributions rendered as a horizontal bar chart with feature names and values.
UI-4	The detail view shall include the customer's last 20 transactions in a timeline view, with the current transaction highlighted.
UI-5	The analyst shall be able to take an action (Confirm Fraud, Dismiss, Request Step-Up, Escalate) with a single click from the detail view, followed by a mandatory reason/notes field before submission.
UI-6	The console shall display the current case queue length and an estimated SLA countdown (time remaining before the oldest unassigned case breaches the configured SLA) in the page header.
UI-7	The console shall automatically refresh the case queue every 30 seconds (configurable) and shall display a real-time notification banner when a new CRITICAL-risk case arrives.

G.2 Risk Officer Rule Configuration Console

Req ID	Requirement
UI-8	The rule configuration console shall display all active rules in a table showing: Rule ID, Name, Conditions summary, Threshold values, Severity, Status (ACTIVE/SHADOW/DISABLED), Last Modified, Modified By.
UI-9	Creating or editing a rule shall use a structured form with: dropdown for condition type, numeric input for threshold, AND/OR logic builder for compound rules (RB-4), severity level selector, and a plain-English preview sentence summarizing the rule (e.g., 'Flag if transaction count in 5 minutes exceeds 5 AND device is new').

UI-10	The console shall show, for each active rule, a 7-day rolling metric: transactions flagged by this rule, of which what percentage were confirmed fraud vs. dismissed (to help Risk Officers detect rules generating excessive false positives).
UI-11	Shadow mode activation (RB-6) shall be accessible from the rule edit view as a toggle with a confirmation dialog explaining that the rule will score live traffic but not affect decisions.

G.3 Executive Dashboard

Req ID	Requirement
UI-12	The executive dashboard shall display the following real-time KPI tiles, each showing the current value and a delta vs. the prior equivalent period (day, week, or month): Total Transactions, Fraud Transactions, Fraud Rate (%), Fraud Losses (currency), Amount Saved by System (estimated), False Positive Rate (%), Active Open Cases, High/Critical Risk Accounts.
UI-13	The dashboard shall include a Fraud Trend line chart showing daily fraud rate, fraud count, and transaction volume over the last 30 days, with a secondary Y-axis for volume.
UI-14	The dashboard shall include a channel breakdown bar chart showing fraud rate by channel (Mobile, ATM, POS, Card, QR, Internet, Agency) for the selected period.
UI-15	The dashboard shall include a Top Fraud Types donut chart (pie chart with center total) showing the distribution of confirmed fraud cases by fraud typology (Section A taxonomy).
UI-16	The dashboard shall include a Geographic Fraud Map showing fraud density by country/region using a choropleth visualization.
UI-17	The dashboard shall include an AI Model Performance panel showing, for each deployed model: current AUC-ROC, precision, recall, and a trend sparkline over the last 30 days.
UI-18	All dashboard date ranges shall be configurable (today, last 7 days, last 30 days, last 90 days, custom date range) via a date picker in the dashboard header.

H. Deployment and Infrastructure Requirements

This section specifies the infrastructure and deployment requirements derived from the non-functional requirements in Section 5. The architecture described here is consistent with the event-driven, microservices deployment pattern used in published bank fraud platform architectures [2][10][16].

H.1 Deployment Architecture

Req ID	Requirement
INF-1	The System shall be deployable as a set of containerized microservices (Docker containers) orchestrated by Kubernetes, consistent with the scalable serverless/container-based architecture pattern used in cloud-native fraud detection platforms [2][10].
INF-2	Each detection layer (Rule Engine, ML Scoring Service, AI Ensemble Service) shall be independently deployable and scalable, so that the ML scoring service can be scaled to handle load spikes without scaling the rule engine proportionally.
INF-3	The System shall support horizontal auto-scaling of the ML scoring service based on CPU utilization and/or queue depth metrics.
INF-4	The System shall support a blue/green or canary deployment strategy for ML model updates, enabling the gradual traffic shift from old model to new model required by the shadow deployment workflow (ML-7).

H.2 Event Streaming

Req ID	Requirement
INF-5	Transaction events from all channels shall be ingested via an event streaming layer (Apache Kafka or equivalent managed streaming service) that provides durable, ordered, and at-least-once delivery guarantees.
INF-6	The streaming layer shall be sized to handle peak transaction ingestion rates without message lag exceeding 500ms under P99 load conditions.
INF-7	A dead-letter queue (DLQ) shall capture any transaction events that fail processing, for investigation and reprocessing; no transaction event shall be silently dropped.

H.3 Data Storage

Data Type	Recommended Store	Rationale
Transaction Events (raw)	Object storage (S3-compatible)	Durable, cost-effective long-term retention for model retraining and audit.

Fraud Decision Records	PostgreSQL (relational)	Structured, queryable, with transactional integrity for audit log requirements.
Cases	PostgreSQL (relational)	Relational case workflow with foreign keys to decisions and transactions.
Customer Risk Profiles	Redis (in-memory) + PostgreSQL (durable)	Redis for sub-10ms profile lookup during real-time scoring; PostgreSQL as source of truth.
Feature Store (rolling aggregates)	Redis (in-memory)	Sub-100ms feature retrieval requirement (FE-1) requires in-memory store.
ML Model Artifacts	MinIO or S3-compatible object storage	Versioned storage for model binaries referenced by the model registry (ML-6).
Search / Analytics Index	Elasticsearch / OpenSearch	Full-text and aggregation queries for analyst investigation and dashboard data [2].

H.4 Monitoring and Observability

Req ID	Requirement
INF-8	The System shall expose real-time operational metrics (API latency percentiles P50/P95/P99, transaction throughput, decision distribution, error rates) via a Prometheus-compatible metrics endpoint, visualized in Grafana dashboards.
INF-9	The System shall ship structured application logs to a centralized log aggregation system (ELK Stack or equivalent) with sufficient context in each log entry to reconstruct the processing of any individual transaction.
INF-10	ML model performance metrics (score distribution, AUC-ROC on rolling confirmed labels, false-positive rate) shall be tracked in the monitoring stack and shall trigger automated alerts when thresholds defined in ML-8 are breached.
INF-11	The System shall implement distributed tracing (e.g., OpenTelemetry) so that the end-to-end latency of a transaction scoring request can be decomposed into per-service segments for debugging and SLA verification.

H.5 Security and Network

Req ID	Requirement
INF-12	All inter-service communication shall use mutual TLS (mTLS) within the cluster; services shall not accept unauthenticated internal connections.
INF-13	The fraud-check API endpoint (Section 4.2) shall be protected by an API gateway that enforces: rate limiting (to prevent abuse), IP allowlisting or mTLS for channel application callers, and request payload validation before forwarding to downstream services.

INF-14	All secrets (API keys, database credentials, encryption keys) shall be managed via a secrets management service (e.g., HashiCorp Vault or a cloud-native equivalent); hard-coded credentials are prohibited.
INF-15	The System's infrastructure configuration shall be managed as code (IaC, e.g., Terraform or CloudFormation) and stored in version control, enabling reproducible environment provisioning and change auditing.

I. Testing Requirements

This section specifies what must be tested and how. It does not replace a separate Test Plan document, but establishes the minimum testing obligations that must be satisfied before the System is deployed to any production channel.

I.1 Unit Testing

Req ID	Requirement
TST-1	Each rule in the rule engine shall have at least one positive test case (transaction that should fire the rule) and one negative test case (similar transaction that should not fire the rule), automated and runnable as part of the CI pipeline.
TST-2	Feature engineering functions (Section E) shall be unit-tested with known inputs and expected outputs, including edge cases: first transaction (no history), zero-amount transaction, maximum-length beneficiary ID.
TST-3	The AI ensemble combination logic (AI-1 through AI-4) shall be unit-tested with synthetic rule/ML score combinations covering all resolution policy branches (AI-4).

I.2 Integration Testing

Req ID	Requirement
TST-4	End-to-end integration tests shall replay a set of representative historical transactions through the full detection pipeline (rules → ML → ensemble → decision) and assert that the final decision matches the expected outcome for each test case.
TST-5	Integration tests shall cover all use cases defined in Section B (UC-01 through UC-05), verifying correct system behavior for each flow including alternative/exception flows.
TST-6	The fraud-check API integration test suite shall include tests for: missing optional fields, malformed JSON, oversized payloads, and requests from unauthorized callers — verifying that the API gateway handles each correctly (INF-13).

I.3 ML Model Validation Testing

Req ID	Requirement
TST-7	Every candidate model promoted from SHADOW to PRODUCTION status must have passed the evaluation criteria in ML-5 on a held-out test set whose date range does not overlap the training data (time-based split).
TST-8	ML model validation shall include adversarial test cases: transactions crafted to resemble known fraud patterns but modified to evade simple threshold rules. At minimum, one adversarial test case per fraud type in Section A.

TST-9	The model validation report shall document the confusion matrix, precision/recall at multiple operating thresholds, and the AUC-ROC curve, retained in the model registry entry (ML-6).
TST-10	SHAP explanations (AI-5) shall be spot-checked on a sample of held-out fraud cases to verify that the top-ranked features are plausible given domain knowledge; implausible top features (e.g., 'transaction_id' ranking as the highest-importance feature) indicate data leakage and shall block promotion.

I.4 Performance and Load Testing

Req ID	Requirement
TST-11	A load test shall be conducted simulating the bank's peak transaction rate, sustained for a minimum of 30 minutes, asserting that P99 end-to-end latency remains within the budget specified in NF-1 and that no errors occur above the configured error-rate threshold.
TST-12	A failover test shall simulate fraud engine unavailability (simulated by taking the ML scoring service down) and verify that the fallback behavior specified in NF-11 is correctly applied by the channel application integration.
TST-13	A scalability test shall demonstrate that the System auto-scales correctly (INF-3) when transaction ingestion rate is increased by 3× over baseline.

I.5 Security Testing

Req ID	Requirement
TST-14	A penetration test or automated SAST/DAST scan shall be conducted on the fraud-check API and case management console before production deployment, targeting at minimum the OWASP API Security Top 10 vulnerability categories.
TST-15	The mTLS configuration (INF-12) shall be verified by attempting unauthenticated inter-service calls and confirming they are rejected.
TST-16	PCI DSS scope validation shall confirm that cardholder data does not flow through or persist in any System component not covered by the PCI DSS controls documented in Section 5.2 (NF-3).

I.6 Fraud Type Coverage Testing

For each fraud type in Section A, the test plan shall include at minimum one test scenario simulating a real transaction sequence that should result in detection by the System. The table below defines the minimum test coverage matrix.

Fraud Type (Section A)	Minimum Test Scenario	Expected Outcome
Account Takeover (ATO)	Login from new device + new ISP + new country, followed by new beneficiary	CRITICAL risk, REJECT or FREEZE decision.

	addition and high-value transfer within 10 minutes.	
Card-Not-Present (CNP)	Transaction at high-risk MCC from a card whose recent transactions are geographically inconsistent with the merchant.	HIGH risk, HOLD decision.
SIM Swap	Mobile number change recorded, followed within 24h by login with OTP-based auth and new high-value beneficiary.	HIGH or CRITICAL, HOLD or REJECT.
Impossible Travel	Two authenticated transactions from cities > 5,000 km apart within 1 hour.	CRITICAL (rule-based immediate flag), HOLD or REJECT.
Velocity Fraud	6 transactions within 60 seconds to different merchants, each below the single-transaction threshold.	HIGH risk, HOLD decision (velocity rule fires).
Dormant Account	Account with no transactions in 180 days initiates a transfer equal to 10× the pre-dormancy average transaction amount.	HIGH risk, HOLD decision.
Money Mule / Layering	Account receives a large inbound transfer and immediately initiates 3 outbound transfers to different beneficiaries summing to the same amount.	HIGH or CRITICAL, HOLD; graph analysis flags structuring pattern.

J. Performance Benchmarks and Acceptance Criteria

This section defines the measurable acceptance criteria that the System must demonstrate before being approved for production operation. These criteria are based on published performance results from comparable deployed systems, adjusted to be realistic for an initial production deployment.

Note on published benchmark figures: the values below are informed by results reported in published research on real fraud datasets [19][29][30][33], but they should be validated against the organization's own historical data before being adopted as contractual commitments. They are labeled as targets, not guarantees.

Criterion	Minimum Acceptable (MVP / Phase 1)	Target (Full Platform)	Basis
Fraud Detection Rate (Recall)	> Existing rule-only baseline (to be measured pre-launch)	≥ 85% on held-out test set	Published ensemble results report 85–98% recall on real-world online banking fraud data [19][29][30].
False Positive Rate	< Existing rule-only baseline	< 5% (95%+ of held transactions are confirmed fraud)	Reducing false positives is a primary motivation for replacing rule-only detection [12][13].
AUC-ROC (ML models)	≥ 0.85 on held-out test set	≥ 0.92	Hyper-ensemble models evaluated on European bank transaction data reported individual model accuracies of 0.91–0.97 [33].
API Latency (P99)	< 1,000 ms end-to-end	< 300 ms end-to-end	Amazon Fraud Detector-based architectures return predictions 'with minimal latency' designed for synchronous transaction authorization [2].
System Availability	≥ 99.5%	≥ 99.9%	Standard for production payment-adjacent infrastructure.

Case SLA Compliance	≥ 80% of HOLD cases reviewed within 4 hours	≥ 95% within 2 hours	Analyst SLA target to be set by the fraud operations team; these are illustrative starting points.
Feature Store Latency (P99)	< 200 ms	< 100 ms	FE-1 requirement; necessary to stay within overall API latency budget.
Model Retraining Cycle	Monthly minimum	Continuous / triggered by drift detection	ML-8/ML-9 requirement; drift-triggered retraining is the pattern in mature production systems [2][17].

K. Glossary

This glossary defines terms used in this SRS that may be unfamiliar to non-technical stakeholders, and provides consistent definitions for technical terms that have multiple meanings in different contexts.

Term	Definition
Anomaly Detection	Unsupervised or semi-supervised ML approach that identifies data points deviating significantly from learned normal patterns, without requiring labeled fraud examples. Used in the autoencoder and Isolation Forest components (Section F.4/F.5).
AUC-ROC	Area Under the Receiver Operating Characteristic Curve. A model evaluation metric that measures the model's ability to discriminate between fraud and non-fraud across all possible classification thresholds. A value of 1.0 indicates perfect discrimination; 0.5 indicates random chance.
Behavioral Analytics	Analysis of patterns in how a customer normally behaves (transaction timing, amounts, devices, locations) to detect deviations that may indicate account compromise or unauthorized activity.
Class Imbalance	The condition, universally present in fraud detection, where legitimate transactions vastly outnumber fraudulent ones (often 100:1 to 10,000:1). Standard ML evaluation metrics such as accuracy are misleading under severe class imbalance.
Dead-Letter Queue (DLQ)	A message queue that receives events that failed to be processed normally, preventing silent data loss and enabling investigation and reprocessing.
Decision Trail	The complete logged record of why a specific fraud decision was made: which rules fired, the ML model score(s), the composite risk score, the risk band, and the SHAP feature attributions. Required for audit and dispute resolution (AI-6, NF-9).
Device Fingerprint	A hash or structured identifier derived from device attributes (operating system, browser, screen resolution, hardware characteristics) that enables the System to recognize a returning device without relying on cookies or device IDs.
Drift (Model Drift)	The degradation of a deployed model's performance over time as the real-world data it scores diverges from the distribution of the data it was trained on. Fraud patterns evolve, causing deployed models to become less accurate unless retrained.
Ensemble / Ensemble Learning	A technique that combines the predictions of multiple individual models to produce a final prediction that is more accurate and robust than any single model [9][14][29].
F1-Score	The harmonic mean of precision and recall. A balanced metric used when both false positives and false negatives have significant costs. $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$.

Feature Store	A centralized system that computes, stores, and serves ML features consistently across model training and production scoring pipelines, preventing training/serving skew (ML-2, FE-1 through FE-3).
Haversine Distance	The great-circle distance between two points on a sphere (Earth), computed from their latitude and longitude coordinates. Used for impossible-travel detection.
Impossible Travel	A fraud signal where two authenticated events (logins or transactions) originate from geographic locations that cannot be reached in the elapsed time between events at any realistic travel speed.
Isolation Forest	An unsupervised anomaly detection algorithm that identifies anomalies by measuring how easily a data point can be isolated from the rest of the data using random tree partitions. Points that are easily isolated are more anomalous.
LSTM	Long Short-Term Memory — a type of recurrent neural network (RNN) designed to model sequential data, capable of learning patterns across sequences of transactions (Section F.3).
Meta-Learner	In stacking ensemble learning, the model that takes the predictions of base learners as its input and produces the final prediction. Often a simpler model such as logistic regression or random forest [30][31].
MCC (Merchant Category Code)	ISO 18245 standard codes that classify businesses by the products or services they primarily sell. Used as a fraud-risk signal since certain merchant categories are associated with higher fraud rates.
Precision	Of all transactions the model flags as fraud, what fraction actually are fraud. High precision = few false positives.
Recall	Of all transactions that actually are fraud, what fraction the model correctly flags. High recall = few missed frauds (false negatives). Recall is the critical metric in fraud detection since missed fraud results in direct financial loss.
SHAP (SHapley Additive exPlanations)	A game-theoretic method for explaining individual ML model predictions by attributing the model's output to each input feature. Provides a human-interpretable explanation of why a model assigned a particular fraud score [7].
Structuring	An AML technique where large sums are broken into multiple smaller transactions (each below reporting thresholds) to avoid triggering mandatory AML reports. Detected by graph-based pattern analysis (GR-2).
Training/Serving Skew	A production ML failure mode where the features computed during model training differ from those computed during real-time scoring, causing the model to perform significantly worse in production than on test data. Prevented by the Feature Store requirement (ML-2).

9. References

The following sources were consulted in preparing this specification. Where this document states a fact, figure, or named practice as coming from real-world industry or regulatory sources, it corresponds to one of the entries below.

- [1] IEEE Std 830-1998, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Computer Society, 1998 (superseded by ISO/IEC/IEEE 29148:2011). <https://standards.ieee.org/ieee/830/1222/>
- [2] A. Mohan, “Banking Fraud Detection with Machine Learning and Real-time Analytics on AWS,” AWS for Industries Blog, Amazon Web Services, Nov. 13, 2023. <https://aws.amazon.com/blogs/industries/banking-fraud-detection-with-machine-learning-and-real-time-analytics-on-aws/>
- [3] Financial Crimes Enforcement Network (FinCEN) and the U.S. Bank Secrecy Act, as referenced in: “Advanced fraud detection using machine learning models: enhancing financial transaction security,” arXiv:2506.10842, 2025. <https://arxiv.org/pdf/2506.10842>
- [4] “AI-driven transaction monitoring: The future of AML, fraud detection, and risk management in banking,” Robotics & Automation News, Feb. 19, 2026. <https://roboticsandautomationnews.com/2026/02/19/ai-driven-transaction-monitoring-the-future-of-fraud-and-risk-management-in-global-banking/98981/>
- [5] “US FinTech Compliance: PCI-DSS, KYC & AML Explained,” NewAge Systit, Apr. 20, 2026. <https://newagesysit.com/blog/us-fintech-security-regulatory-compliance-pci-dss-kyc-aml-explained/>
- [6] PCI Security Standards Council, PCI DSS v4.0, 2022; FinCEN SAR requirements, 2021; as cited in “Advanced fraud detection using machine learning models: enhancing financial transaction security,” Section 5.3, arXiv:2506.10842. <https://arxiv.org/pdf/2506.10842>
- [7] S. M. Lundberg and S.-I. Lee, SHAP (SHapley Additive exPlanations), referenced via “Explainable ensemble machine learning for financial transaction fraud detection: Insights from XGBoost and deep neural networks,” ScienceDirect, 2026. <https://www.sciencedirect.com/science/article/pii/S2405918826000206>
- [8] “Machine Learning Approach for Fraud Detection System in Financial Institution: A Web Base Application,” ResearchGate, Apr. 2024. https://www.researchgate.net/publication/380174951_Machine_Learning_Approach_for_Fraud_Detection_System_in_Financial_Institution_A_Web_Base_Application
- [9] “Enhancing credit card fraud detection with a stacking-based hybrid machine learning approach,” PMC, National Center for Biotechnology Information. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12453863/>
- [10] “AI Fraud Detection in Banking 2026 Guide,” Emburse, May 15, 2026. <https://www.emburse.com/resources/ai-fraud-detection-in-banking>
- [11] “Enhancing Fraud Detection in Banking with Rule-Based Decision Engines,” Higson Blog, Jun. 20, 2024. <https://www.higson.io/blog/enhancing-fraud-detection-in-banking-with-rule-based-decision-engines>

- [12] “Fraud Detection Using Machine Learning in Banking,” Tookitaki Compliance Hub. <https://www.tookitaki.com/compliance-hub/fraud-detection-using-machine-learning-in-banking>
- [13] “Fraud Detection Using Machine Learning vs. Rules-Based Systems,” Fraud.net Resources, Apr. 1, 2026. <https://www.fraud.net/resources/fraud-detection-using-machine-learning-vs-rules-based-systems>
- [14] “Machine Learning for Financial Fraud Detection: Use Cases & Best Practices,” Svitla Systems, Feb. 13, 2026. <https://svitla.com/blog/machine-learning-for-financial-fraud-detection/>
- [16] “AI Fraud Detection in Banking 2026 Guide” (hybrid deployment and staged-rollout discussion), Emburse, May 15, 2026. <https://www.emburse.com/resources/ai-fraud-detection-in-banking>
- [17] A. Mohan, “Banking Fraud Detection with Machine Learning and Real-time Analytics on AWS” (continuous retraining discussion), AWS for Industries Blog, Nov. 13, 2023. <https://aws.amazon.com/blogs/industries/banking-fraud-detection-with-machine-learning-and-real-time-analytics-on-aws/>
- [19] “A High-Recall Cost-Sensitive Machine Learning Framework for Real-Time Online Banking Transaction Fraud Detection,” arXiv:2601.07276. <https://arxiv.org/pdf/2601.07276>
- [21] General Data Protection Regulation (GDPR) data-protection principles, as referenced in: “Expertise: Security and compliance,” Digital Bank Expert, Apr. 1, 2026. <https://digitalbankeexpert.com/security-and-compliance/>
- [29] “Explainable ensemble machine learning for financial transaction fraud detection: Insights from XGBoost and deep neural networks,” ScienceDirect, 2026. <https://www.sciencedirect.com/science/article/pii/S2405918826000206>
- [30] “Enhancing credit card fraud detection with a stacking-based hybrid machine learning approach,” NCBI/PMC, 2025. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12453863/>
- [31] “Enhancing credit card fraud detection with a stacking-based hybrid machine learning approach,” PMC (full text, methodology and F1-score result), 2025. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12453863/>
- [33] “Building Resilience in Banking Against Fraud with Hyper Ensemble Machine Learning and Anomaly Detection Strategies,” SN Computer Science, Springer Nature, May 17, 2024. <https://link.springer.com/article/10.1007/s42979-024-02854-w>